Database Management Systems

Student Hub | Original study summary | For revision and self-study

Course

CS302 | Student Hub Original Revision Guide

1. Relational model

A relation is represented as rows and columns. A primary key identifies a row; a foreign key references a key in another relation. Choose keys that are unique, stable, and non-null.

2. ER design

Identify entities, attributes, and relationships before creating tables. Resolve many-to-many relationships with a junction table. Record cardinality and optionality so the schema reflects the real rules.

3. Normalization

First normal form stores atomic values and avoids repeating groups. Second normal form removes partial dependencies on part of a composite key. Third normal form removes non-key dependencies on other non-key attributes.

4. SQL essentials

SELECT chooses columns; WHERE filters rows; JOIN combines related tables; GROUP BY aggregates groups; HAVING filters groups; ORDER BY sorts results. Use parameters for values supplied by users.

5. Transactions

A transaction groups changes into one logical unit. ACID means atomicity, consistency, isolation, and durability. Commit saves the unit; rollback cancels uncommitted changes.

6. Indexes and constraints

Indexes can speed up filters and joins but consume storage and add write cost. Use NOT NULL, UNIQUE, CHECK, and foreign-key constraints to enforce valid data close to the database.

Study checklist

Sketch an ER diagram, map it to tables, mark keys, normalize a small example, then write a parameterized SELECT with a join and aggregate.

Database Management Systems

Student Hub | Original study summary | For revision and self-study

Practice schema

Imagine Student(student_id, name) and Enrollment(student_id, course_id, grade). The pair (student_id, course_id) can be a composite key; student_id also references Student.

Practice prompts

1) Why is a foreign key useful? 2) Which clause filters aggregate groups? 3) What does rollback do? 4) Why should user input be bound as a query parameter?

Suggested answers

1) It preserves referential integrity. 2) HAVING. 3) It discards uncommitted changes in the transaction. 4) It keeps data separate from SQL syntax and helps prevent injection.

SQL habit

Select only the columns you need. Use explicit JOIN conditions, handle NULL deliberately, and inspect query plans when performance matters.

Further reading

The PostgreSQL SQL tutorial and SQLBolt lessons are linked from the Student Hub Course Materials page.